

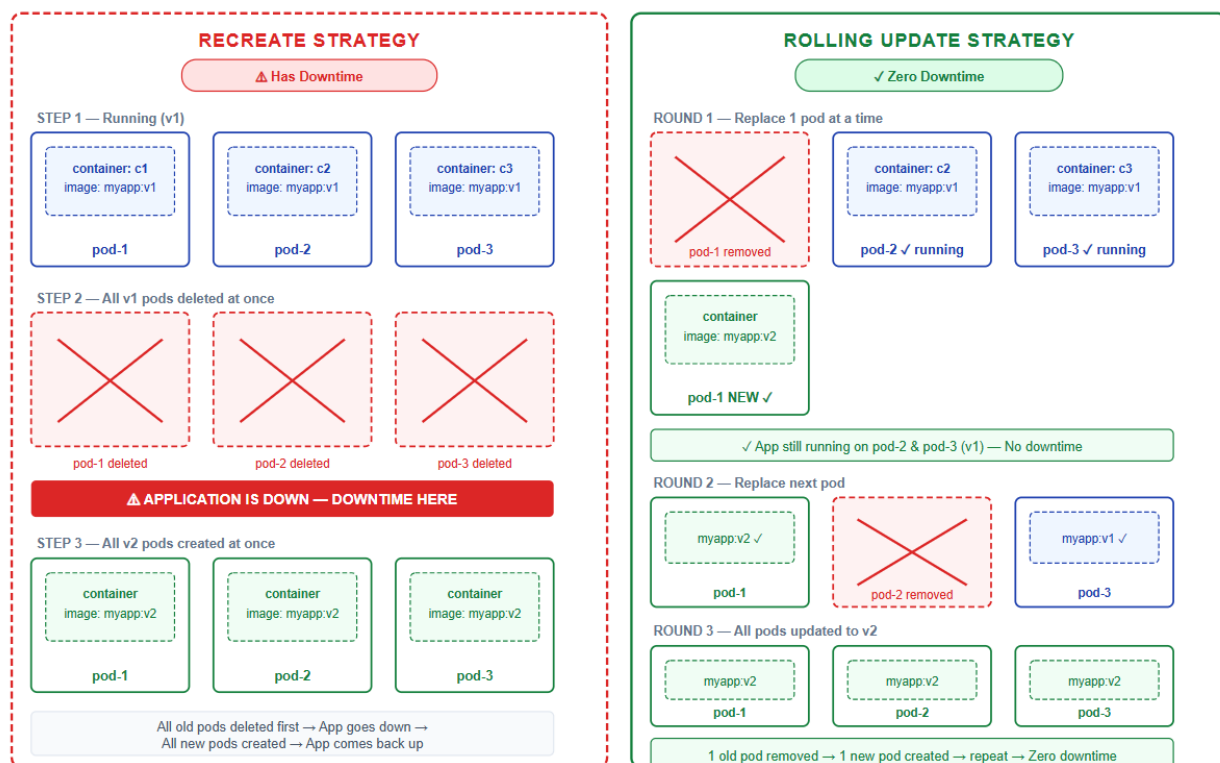
What all Deployment strategies used in Kubernetes?

i. Recreate strategy:

deleting all pods of old version at once & creating new pods with new version, this strategy we can observe application downtime.

ii. Rolling update strategy(default type):

- it will delete 1 old version of a pod & bring 1 new version of a pod
- this strategy we will not see any downtime.
- this is the default strategy used in deployments.



What is maxSurge and maxUnavailable in k8s deployments?

Both are additional features that can be used in deployments (rolling update) to ensure HA & lesser downtime during rolling update process

maxsurge means k8s will create **few extra pods temporarily** to speed up rolling update process & for HA.

maxsurge can be specified with fixed a number (like 1 to 10) **or** a percentage (like 30%).

maxUnavailable will ensures the safety of rolling updates.

maxUnavailable specifies how many **old versions of pods** can be **"not available"** (getting deleted) at the same time during a rolling update.

helps in preventing too much downtime.

Example: if we have a deployment with 10 replicas then *during rolling update process*,

==> if **maxSurge** is set to 4 ==> K8s will create up to 4 extra Pods (14 total),

==> if **maxUnavailable** is set to 2 ==> k8s make sure only 2 Pod can be unavailable at any given time.

in deployment manifest-file under spec section add like below:

```
spec: #deployment-spec
  replicas: 10
  strategy:
    rollingUpdate:
      maxSurge: 4
      maxUnavailable: 2
    type: RollingUpdate
```

DEPLOYMENT ==creates==> REPLICASETS ==creates==> POD ==creates==>
container(application)

Updating applications to newer versions in K8S:

in containerized applications: if we are updating our application to next version means, we are creating new docker image with newer tag, we use the updated docker image in our manifest files

example: below are versions of nginx application, where **image tag** will gets updated with each version changes

nginx:1.24 → nginx:1.26 → nginx:1.28 → nginx:1.30

Create deployment with

- Image - nginx:1.24 & 3 replicas
- Labels – author as bharath & environments: staging
- Nginx runs on port 80 & Container name as nginx-c1

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      author: bharath
      environments: staging
  template:
    metadata:
      name: nginx-deployment-v1
      labels:
        author: bharath
        environments: staging
    spec:
      containers:
      - name: nginx-c1
        image: nginx:1.24
        ports:
        - containerPort: 80
...
```

Commands to update & rollback Deployments in k8s

1) To update deployment to new version

```
kubectl set image deployment <deployment-name> <container-name>=<image-to-be-updated>:<tag> --record=true
```

2) To check the Deployments revision history

```
kubectl rollout history deployment <deployment-name>
```

3) To Rollback deployment to its previous version

```
kubectl rollout undo deployment <deployment-name>
```

Case study:

Let's assume your company we have created Zomato like application

which will have initial versions like ==> **acecloudacademy/zomato-clone-image:v1**

Next versions(images) would be like:

images==> acecloudacademy/zomato-clone-image:v1 → acecloudacademy/zomato-clone-image:v2 → acecloudacademy/zomato-clone-image:v3 → (keeps.. on)

Case 1: Deploy a new application version (new image ==> **acecloudacademy/zomato-clone-image:v1**) using rolling deployment strategy?

Create deployment name **zomato-deployment**

- Image - acecloudacademy/zomato-clone-image:v1
- Replicas – 2
- Labels – type as food-delivery-app
- Container name as c1

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: zomato-deployment
  labels:
    type: food-delivery-app
spec:
  replicas: 2
  selector:
    matchLabels:
      type: food-delivery-app
  template:
    metadata:
      name: zomato-pod
      labels:
        type: food-delivery-app
    spec:
      containers:
      - name: c1
        image: acecloudacademy/zomato-clone-image:v1
        ports:
        - containerPort: 3000
...
```

Rolling Update strategy is default strategy type used in Kubernetes

```
root@k8s-master:~#
root@k8s-master:~# kubectl get deploy
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
zomato-deployment   2/2     2            2           7s
root@k8s-master:~#
```

Case 2: after 3 months to give new features to end-users, then we will create new image ==> **acecloudacademy/zomato-clone-image:v2**

```
root@k8s-master:~# #Check current version image
root@k8s-master:~# kubectl describe deployment zomato-deployment | grep -i image
      Image:          acecloudacademy/zomato-clone-image:v1
root@k8s-master:~#
root@k8s-master:~#
root@k8s-master:~# Syntax: TO UPDATE DEPLOYMENT TO NEWER VERSION
root@k8s-master:~# kubectl set image deployment <deployment-name> <container-name>=<image-to-be-updated>:<tag>
root@k8s-master:~#
```

```
root@k8s-master:~# Update deployment to use new V2 image
root@k8s-master:~# kubectl set image deployment zomato-deployment c1=acecloudacademy/zomato-clone-image:v2 --record=true
Flag --record has been deprecated, --record will be removed in the future
deployment.apps/zomato-deployment image updated
root@k8s-master:~#
root@k8s-master:~# # Check Rollout history of deployment
root@k8s-master:~# kubectl rollout history deployment zomato-deployment
deployment.apps/zomato-deployment
REVISION  CHANGE-CAUSE
1          <none>
2          kubectl set image deployment zomato-deployment c1=acecloudacademy/zomato-clone-image:v2 --record=true
root@k8s-master:~#
```

Case3: after another 3 months to give few more additional features to end-users we will create new image ==> **acecloudacademy/zomato-clone-image:v3**

```
root@k8s-master:~#
root@k8s-master:~# # Update application to v3 image
root@k8s-master:~# kubectl set image deployment zomato-deployment c1=acecloudacademy/zomato-clone-image:v3 --record=true
Flag --record has been deprecated, --record will be removed in the future
deployment.apps/zomato-deployment image updated
root@k8s-master:~#
root@k8s-master:~# # Check deployment Rollout History
root@k8s-master:~# kubectl rollout history deploy zomato-deployment
deployment.apps/zomato-deployment
REVISION  CHANGE-CAUSE
1          <none>
2          kubectl set image deployment zomato-deployment c1=acecloudacademy/zomato-clone-image:v2 --record=true
3          kubectl set image deployment zomato-deployment c1=acecloudacademy/zomato-clone-image:v3 --record=true
root@k8s-master:~#
root@k8s-master:~#
```

Case4: After deploying **acecloudacademy/zomato-clone-image:v3** if we face issue in production then we need to rollback to immediate previous version

```
root@k8s-master:~#
root@k8s-master:~# # Check app has been updated to V3
root@k8s-master:~# kubectl describe deploy zomato-deployment | grep Image
      Image:          acecloudacademy/zomato-clone-image:v3
root@k8s-master:~#
root@k8s-master:~#
root@k8s-master:~# # If something goes wrong with app, we need to rollback to immediate previous version (image - v2)
root@k8s-master:~# kubectl rollout undo deploy zomato-deployment
deployment.apps/zomato-deployment rolled back
root@k8s-master:~# # Check deployment history for rollback
root@k8s-master:~# kubectl rollout history deploy zomato-deployment
deployment.apps/zomato-deployment
REVISION  CHANGE-CAUSE
1          <none>
3          kubectl set image deployment zomato-deployment c1=acecloudacademy/zomato-clone-image:v3 --record=true
4          kubectl set image deployment zomato-deployment c1=acecloudacademy/zomato-clone-image:v2 --record=true
root@k8s-master:~# #Verify app rollbacked to v2
root@k8s-master:~# kubectl describe deploy zomato-deployment | grep Image
      Image:          acecloudacademy/zomato-clone-image:v2
root@k8s-master:~#
```

Kubectl rollout options:

1.How to check a deployments history ?

```
kubectl rollout history deployment <deployment_name>
```

```
kubectl rollout history deployment nginx-deployment
```

2. How to rollback deployment to previous version?

```
kubectl rollout undo deployment <deployment_name>
```

(or) < to rollback to specific version >

```
kubectl rollout undo deployment <deployment_name> --to  
revision=<revision_number>
```

```
kubectl rollout undo deployment nginx-deployment --to-revision=1
```

Assignment:

deploy a new application version again (from nginx:1.26.0 , upgrade it to nginx:1.28.0 & nginx:1.30.0 & finally rollback) using rolling deployment strategy?

Service Object:

- ❖ Service object is used to expose our application (port mapping)
- ❖ Service object will provide a **static ip address** to our application running in pods.
- ❖ Service object will distribute incoming Traffic equally to all pods.

If we don't use service

- If a pod goes down & new pod comes up, pods-ip address will get changed. then pod to pod communication will be difficult
- Traffic cannot be equally distributed to all replicas

A **Service** in Kubernetes is used to:

1. Provide stable IP & DNS name to Pods:

service gives a stable DNS name and IP address to access a set of pods

2. Load balancing:

It automatically distributes incoming traffic across multiple pod replicas, so that no particular pod will get more traffic requests.

3. Simplifies Pod communication:

It allows pods to communicate with each other or with external clients(browser) without needing to know pod IPs.

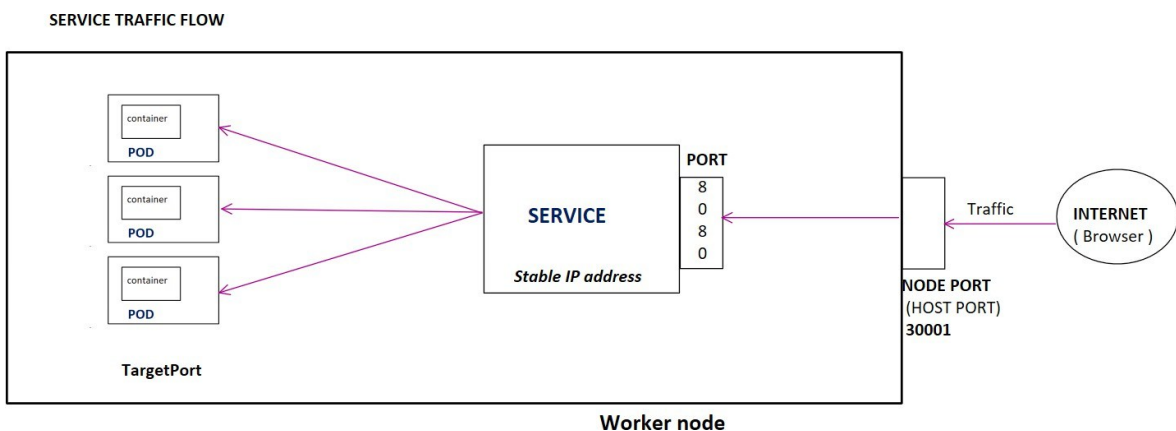
4. Service discovery:

Pods can find other pods using simple names (DNS)

No need to worry about changing Ips of pods

Service Object we use 3 ports in service manifest file, which are

1. **target port** - It is Port inside the container (where application actually runs)
2. **port** - Refers to **service** reference port.
3. **NodePort** - Refers to host machine (worker node) port to make it accessible from external network.



*** important interview question ***

What are different types of Service & difference between those ?:

1. ClusterIP (default):

- ❖ It exposes the service within the Kubernetes cluster only.
- ❖ ClusterIP is used when we want the pods in the cluster to communicate with each other and not with external network(from internet or browser).
- ❖ This is default type of service object used in kubernetes.
- ❖ using clusterip type service pod-to-pod communication will happen within the cluster

2. NodePort:

- ❖ It exposes the service both in and outside the cluster
- ❖ It exposes the service on each Worker Node's IP at a static port (i.e., which is called NodePort).
- ❖ nodePort can be used, if we want to access the pods from an external network (Internet or browser).
- ❖ NodePort must be within the range from 30000-32767

3. LoadBalancer:

- ❖ It exposes the service both in and outside the cluster, it's the **most preferred** way of exposing a service in k8s.
- ❖ It exposes the service externally using cloud providers load balancer (AWS service - ELB-> Elastic load balancers).
- ❖ whenever the LoadBalancer service gets created it will also automatically create NodePort and ClusterIP services.

**Create a deployment using Jenkins/jenkins image with 2 replicas & label type: cicd
Expose the application**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: jenkins-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      type: cicd
  template:
    metadata:
      labels:
        type: cicd
    spec:
      containers:
        - name: c1
          image: jenkins/jenkins
          ports:
            - containerPort: 8080
...
```

To expose the jenkins pods created as a part of above deployment ?

Create a service object of type nodePort & expose the application

- jenkins container runs on port 8080
- assign service port also as 8080
- open nodeport in 30001
- use selector to find pods with lables *type: cicd*

vi my-jenkins-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: jenkins-service
spec:
  type: NodePort
  selector:
    type: cicd
  ports:
    - port: 8080          #service objects port number
      targetPort: 8080    #jenkins-container port number
      nodePort: 30001     #workernode-port number
  ...
```

Important Note on service:

- ❖ service object will never creates pods.
- ❖ service object **searches for all pods** which have labels mentioned under **selector** keyword of service manifest file & expose those pods.
in above example it will look for pods which have label **type** as **cicd**, as jenkins pods have those labels (*type: cicd*), it will expose all the jenkins pods on nodeport 30001.

How to access application from gui?

http://<public_ip_of_any_node>:<nodePort_portnumber>
http://3.90.88.139:30001

deploying zomato-like application as k8s deployment

NotE: before starting this assignment delete all pods /deployments present in your cluster, as this application needs more resources

Create deployment named -- zomato deployment, replicas 1 , image
acecloudacademy/zomato-clone-image:v1

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: zomato-deployment
  labels:
    type: food-delivery-app
spec:
  replicas: 2
  selector:
    matchLabels:
      type: food-delivery-app
  template:
    metadata:
      name: zomato-pod
      labels:
        type: food-delivery-app
    spec:
      containers:
      - name: my-zomato-container
        image: acecloudacademy/zomato-clone-image:v1
        ports:
        - containerPort: 3000
...

```

Create a service of type nodeport to expose zomato application

```
...
apiVersion: v1
kind: Service
metadata:
  name: zomato-service
spec:
  type: NodePort
  selector:
    type: food-delivery-app #finds & exposes all pods with this labels
  ports:
    - port: 3000          #this is service objects refernce port
      targetPort: 3000 #container port ==> zomato listening port on container
      nodePort: 30002  #this is node port assigning to all worker node
...
```

create service, once service matches pods using lables & slector & pods get exposed, access from GUI for zomato application

Assignment:

1. acecloudacademy- netflix-clone-app deployment

Create a Kubernetes Deployment for a Netflix clone application using the image acecloudacademy/netflix-clone-app:v1 with **2 replicas**.

Netflix clone application runs on **container port 80**. Expose this Deployment to external users by creating a Service that makes it accessible on **NodePort 31112**.

2. Grafana Dashboard Deployment

Create a Kubernetes Deployment using the **image grafana/grafana** with 2 replicas. Grafana runs on **container port 3000**. Expose it externally using a Service with **NodePort 32000** so users can access the dashboard through a browser.

3. Kibana UI Deployment:

Create a Kubernetes Deployment using the image **kibana** with 1 replica.

Kibana runs on **container port 5601**.

Expose it externally using a Service with **NodePort 32002** to allow browser-based access to the UI